

TOULOUSE INP N7

Le Vibe Coding

Développer avec l'IA : Guide pratique et retours d'expérience (édition octobre 2026)

TONTON STUDIO
THE FUTURE IS BACK

Qu'est-ce que le vibe coding ?

Le vibe coding (terme lancé par Andrej Karpathy en février 2025) représente une nouvelle approche du développement logiciel où l'intelligence artificielle devient un partenaire actif dans le processus de création. Au lieu de coder ligne par ligne, **vous collaborez avec des modèles de langage** pour conceptualiser, implémenter et itérer rapidement sur vos idées.

Cette méthode transforme la façon dont nous abordons les problèmes techniques : l'IA suggère des solutions, génère du code, identifie des bugs, et vous permet de vous concentrer sur l'architecture et la vision globale.

Mais attention : à l'heure actuelle, il est fortement déconseillé de se lancer dans cette approche pour développer un produit fini sans un **minimum d'expertise technique** et un **maximum de supervision** ! En 2026, Karpathy lui-même réserve le terme au prototypage et parle d'« agentic engineering » pour le travail sérieux.





L'évolution du vibe coding

1

Vibe coding "sauvage"

Multiples itérations avec un LLM : prompt→ bout de code → copiés-collés → prompt….etc. Essais-erreurs sans méthode, gestion du contexte "à la mano", génération de code sans compréhension ni validation.

2

Phase d'apprentissage

Découverte des outils IA spécialisés, premiers prompts structurés, début de méthodologie mais processus encore chaotique.

3

Vibe coding professionnel → agentic engineering

Workflows établis, documentation systématique, gestion rigoureuse du contexte, intégration complète dans les pratiques DevOps. En 2026 : plusieurs agents en parallèle, specs écrites, revue des diffs et tests systématiques.

En 2026, Karpathy remplace le vibe coding par l'agentic engineering pour le travail sérieux

	Vibe coding	Agentic engineering
Posture	On décrit, on accepte, on avance	On pilote des agents comme un tech lead
Spécification	Prompt improvisé	Spec écrite avant de lancer l'agent
Revue	On regarde si « ça marche »	Relecture de chaque diff
Vérification	Essais à la main	Tests et évaluations automatiques
Usage	Prototype, projet perso	Logiciel livré et maintenu

Définition de Karpathy : « la discipline professionnelle qui consiste à coordonner des agents faillibles tout en préservant la justesse, la sécurité, le goût et la maintenabilité ».

Source : A. Karpathy, Sequoia AI Ascent, 30 avril 2026 (citation traduite).

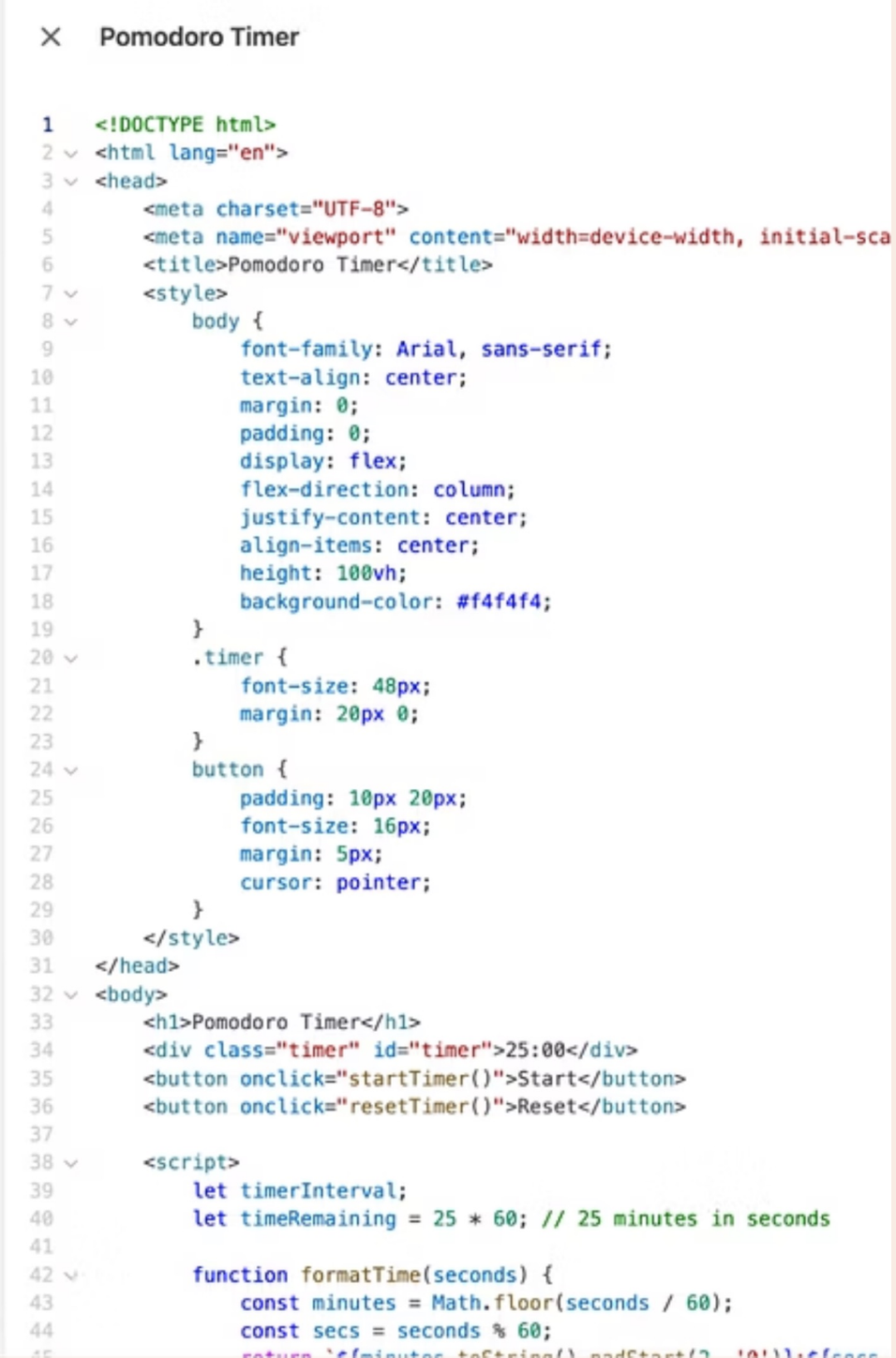
Mon parcours vibe coding : du chaos à la méthode

Le **vibe coding**, c'est l'art de laisser **l'intuition** guider le développement de code, souvent en collaboration étroite avec des outils d'IA. C'est une danse entre la **créativité** et la **logique**, où l'on "ressent" la bonne direction à prendre, plutôt que de suivre un plan rigide.



Étape 1 : les premiers pas avec ChatGPT

Mes débuts ont été marqués par l'expérimentation. Des prompts simples, beaucoup de copier-coller, surtout pour des projets web. C'était excitant, mais le processus était loin d'être structuré.



Étape 2 : l'arrivée de ChatGPT 4o et Canvas

L'intégration de ChatGPT 4o et l'utilisation de Canvas ont marqué un tournant. J'ai commencé à rédiger des **CDC techniques**, à réfléchir à **l'architecture et l'arborescence du projet** et à faire des **sauvegardes régulières**. Une approche plus organisée commençait à émerger.



Étape 3 : la découverte de Lovable et Windsurf

Ces outils ont d'abord semblé magiques, promettant une fluidité incroyable. J'ai été séduit par leur potentiel avec une fenêtre contextuelle paraissant extrêmement bien gérée… Mais j'ai rapidement rencontré des limitations (notamment techniques et financières) qui m'ont poussé à chercher d'autres solutions.

N.B. Windsurf s'appelle Devin Desktop depuis juin 2026.



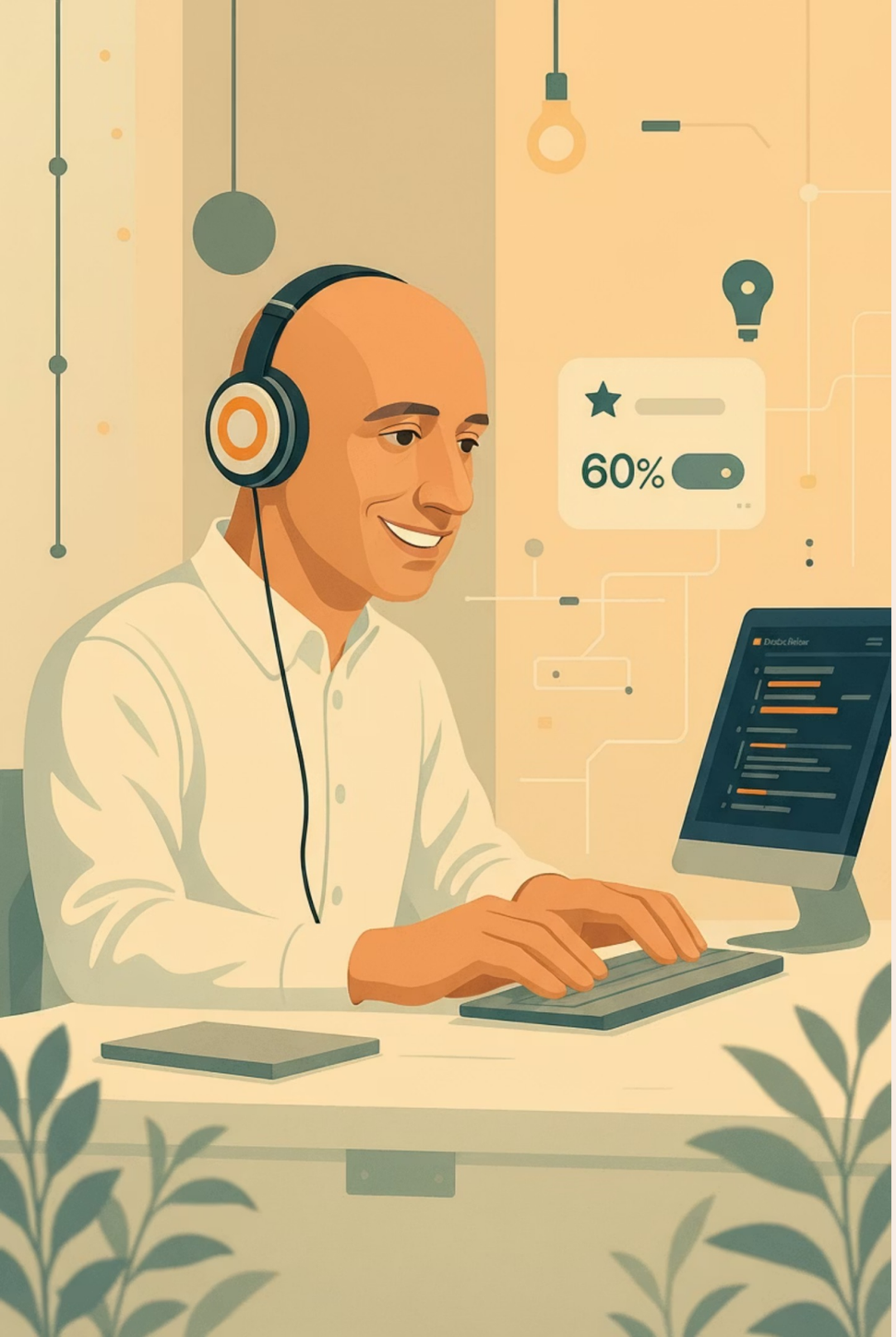
Étape 4 : Claude Desktop et Model Context Protocol

Avec Claude Desktop et les MCP (Model Context Protocol), j’ai commencé à mettre en place un workflow beaucoup plus performant. La gestion du contexte restait laborieuse, mais les résultats étaient significativement meilleurs.



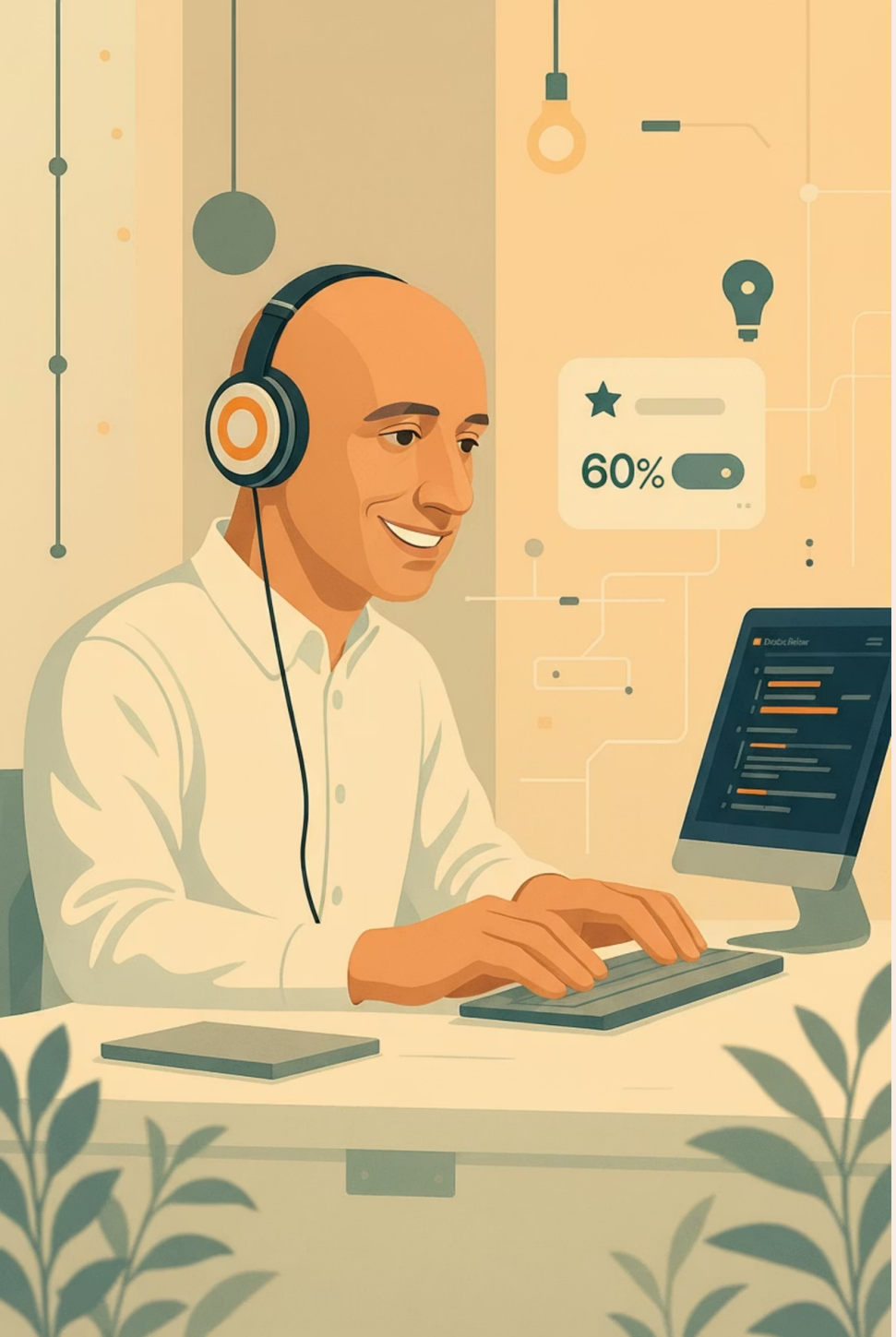
Étape 5 : l'ère de Claude Code

Claude Code a apporté une fluidité et une mémoire contextuelle que je n'avais pas connues auparavant, notamment grâce à son autocompression du contexte, ses possibilités agentiques, sa compatibilité MCP et ses custom commands (devenues des skills). Cela a transformé ma façon de coder avec l’IA !



Étape 6 : vers l'optimisation continue du workflow

Aujourd'hui, mon workflow est bien plus affiné, intégrant des sous-agents spécialisés, des skills, des hooks, des outils optimisant toujours plus mon workflow et une automatisation croissante. Je suis dans une démarche d'amélioration continue, cherchant toujours à booster l'efficacité et la qualité de mon harnais.



Étape 7 (2026) : plusieurs agents en parallèle

Le workflow reste celui d'un solopreneur, mais je pilote maintenant plusieurs agents et plusieurs projets en même temps : sessions parallèles, tâches en arrière-plan, revues automatiques. Le volume abattu explose… et le cerveau chauffe beaucoup plus vite.

Retour d'expérience personnel

⚠️ **Vigilance santé mentale** : Les LLM sont extrêmement convaincants dans leurs réponses, même quand ils se trompent. Cette confiance artificielle peut vous pousser à douter de votre propre jugement.

Le piège de la productivité

Quand l'IA accélère votre travail, il devient tentant de continuer "encore un peu" après avoir terminé une grosse fonctionnalité. **Résistez à cette tentation.**

L'importance des pauses

En milieu professionnel, arrêtez-vous en fin de journée même s'il vous reste des tokens. La fatigue mentale s'accumule plus vite qu'avec du code traditionnel. **Et avec plusieurs agents en parallèle, encore plus !**

Célébrer les victoires

Après avoir résolu un gros problème, prenez le temps de savourer cette réussite. Ne vous précipitez pas sur la tâche suivante immédiatement.

Les études : on se croit plus rapide qu'on ne l'est, et le code généré reste peu sûr

19 % plus lent

METR 2025 : 16 développeurs expérimentés, 246 tâches. Avec l'IA, les tâches ont pris 19 % de temps en plus, alors qu'ils se croyaient 20 % plus rapides.

2026 : pas tranché

Suivi METR (57 devs, 800+ tâches) : tendance à l'accélération (−18 % de temps pour les anciens participants), mais METR juge ces données peu fiables.

45 %

du code généré introduit une faille du Top 10 OWASP (Veracode, 100+ modèles). Ce taux ne s'améliore pas d'une génération de modèles à l'autre.

~20 %

des échantillons de code citent un paquet qui n'existe pas. Des attaquants publient ces noms sur npm ou PyPI : c'est le slopsquatting.

2,74×

plus de failles de sécurité dans les pull requests co-écrites par IA que dans celles écrites par des humains (CodeRabbit, 470 PR).

35 CVE

attribuées à du code généré par IA en mars 2026 seul, plus que sur toute l'année 2025 (Vibe Security Radar, Georgia Tech).

Sources : METR (juillet 2025 ; février 2026) ; Veracode, cité par la Cloud Security Alliance (avril 2026) ; étude USENIX Security 2025 sur les paquets hallucinés ; CodeRabbit (décembre 2025) ; Georgia Tech (avril 2026).

Outils populaires pour le vibe coding

GitHub Copilot

Avantages : intégré à VS Code, JetBrains, etc. ; complétion, mode agent et agent cloud qui ouvre des PR ; gratuit pour les étudiants.

Inconvénients : facturation à l'usage depuis juin 2026 ; choix du modèle limité sur le plan étudiant.

Codex (OpenAI)

Avantages : agent complet : CLI open source, extension IDE, tâches cloud, app intégrée à ChatGPT desktop ; GPT-6.1 Sol au meilleur coût par tâche.

Inconvénients : nécessite au moins ChatGPT Plus (20 $/mois) ; tâches cloud à relire avant de fusionner.

Claude Code

Avantages : agent de référence en terminal, IDE et app ; skills, hooks, sous-agents, MCP ; Opus 5.5 en tête des classements de code (oct. 2026).

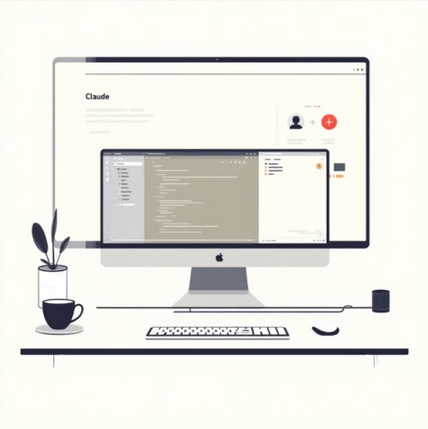
Inconvénients : abonnement Pro (20 $/mois) minimum ; quotas vite atteints sur les longues sessions.

Cursor / Devin Desktop

Avantages : éditeurs complets avec IA et agents cloud en parallèle ; Devin Desktop (ex-Windsurf) pilote aussi Codex ou Claude.

Inconvénients : 20 $/mois ; produits, noms et tarifs qui changent tous les trimestres.

Claude Desktop (Chat, Cowork, Code, Design) ou Claude Code en CLI ?



Claude Desktop (application)

- **Chat :** conversation, artefacts, connecteurs MCP ; seul onglet disponible en gratuit.
- **Cowork :** agent qui mène des tâches multi-étapes sur vos dossiers, dans une VM isolée ; fusionne avec le chat depuis le 16 sept. 2026.
- **Code :** Claude Code en interface graphique : diffs visuels, aperçu de l'appli, sessions parallèles.
- **Design :** maquettes, prototypes et slides sur un canvas, puis relais à Claude Code.
- **Pour qui :** découvrir, prototyper, déléguer sans terminal.



Claude Code (CLI)

- **Même socle :** l'onglet Code et le CLI partagent skills et plugins via votre compte claude.ai.
- **Auto mode :** par défaut depuis août 2026, l'agent valide seul les actions courantes.
- **Extensible :** skills, hooks, plugins, sous-agents en arrière-plan, MCP.
- **Scriptable :** mode headless, CI, tâches planifiées dans le cloud.
- **Pour qui :** le code au quotidien et l'automatisation.

Claude Code : l'agent de code du terminal à l'IDE

Avantages pour le développement :

- **Expérience intégrée :** agent dans le terminal, VS Code, JetBrains ou l'app Desktop ; il lit le projet, édite, lance les tests (le CLI c’est la vie).
- **Édition directe :** Possibilité d'éditer le code directement dans l'interface, sans copier-coller manuel.
- **Prototypage rapide :** idéal pour des cycles de développement rapides, la création de POC (Proof of Concept) ou l'expérimentation de nouvelles idées sur n’importe quelle stack.
- **Accès aux MCP :** bases de données, navigateur, tickets, docs internes ; support MCP complet, au même niveau que Claude Desktop.
- **Mémoire contextuelle dynamique :** système CLAUDE.md et autocompression intelligente du contexte pour maintenir la cohérence sur de longues sessions, plus une mémoire automatique et des skills chargés à la demande.

Inconvénients :

- **Moins de flexibilité :** moins adapté aux workflows de développement hautement personnalisés ou aux exigences de sécurité strictes des environnements d'entreprise (peut rapidement faire d’énormes boulettes).
- **Limitations d'échelle :** peut atteindre ses limites sur des projets avec une base de code très vaste ou des architectures distribuées complexes (comme tous les outils actuels de développement assistés par IA).

Quand l'utiliser :

Pour le développement personnel, les projets open source, l'apprentissage de nouveaux langages/frameworks, et le prototypage rapide d'applications web ou mobiles.

Alternatives gratuites à Claude Code

OpenCode (terminal) (→ lien)

Agent "à la Claude", open-source (MIT, le plus étoilé de GitHub), Linux/Windows ; lit le projet et propose des modifs multi-fichiers ; nécessite une clé API ou un modèle local (75+ fournisseurs).

Antigravity CLI (terminal)

Remplace Gemini CLI depuis le 18 juin 2026 (commande agy) ; offre gratuite avec Gemini 3.5 Flash ; propriétaire, compte Google requis. antigravity.google

Cline (VS Code)

Agent autonome dans VS Code : mode Plan pour cadrer, mode Act pour exécuter, chaque action validée ; modèle au choix (clé API ou local). cline.bot

En secours : Copilot Student / Kiro

Copilot Student : complétions illimitées et 200 crédits IA/mois (education.github.com). Kiro : 50 crédits/mois gratuits (kiro.dev).

Reproduire l'expérience Claude Code sans licence (exemples)

1

Recette Opencode

Installer → Lancer une session → Dialoguer comme avec Claude Code via les modèles proposes → L'outil propose des changements : contrôler et committer.

2

Recette Cline + VS Code

Installer l'extension → Choisir un modèle (OpenRouter, Ollama…) → Mode Plan pour cadrer, puis Act → Valider chaque modification.

3

Recette Antigravity CLI

Installer → Lancer agy et se connecter (compte Google) → Décrire le résultat attendu → Relire les diffs avant d'accepter.

4

Astuces Communes

Tenir un JOURNAL.md (prompts clés, décisions) ; faire des commits courts ; tester à chaque étape ; écrire un AGENTS.md court.

Liens utiles

- Aider
- OpenRouter
- Ollama

AGENTS.md, Skills, MCP : trois couches



AGENTS.md : le projet

Fichier Markdown à la racine du dépôt : commandes de build et de test, conventions, architecture. Lu par Codex, Cursor, Copilot, OpenCode, Antigravity… (Claude Code lit CLAUDE.md, qui peut renvoyer vers AGENTS.md).



Skills : le savoir-faire

Dossier contenant un SKILL.md (nom + description) et des scripts ; l'agent ne le charge que si la tâche l'exige. Standard ouvert, portable d'un outil à l'autre.



MCP : les outils

- Connecte l'agent à des outils et données externes :
- base de données, API, navigateur,
- tickets, documentation interne.

Couche d'intégration, pas de comportement.

À retenir :

- Écrivez AGENTS.md à la main et court : générés par un LLM, ces fichiers baissent le taux de réussite et coûtent 20 % de plus.
- AGENTS.md : https://agents.md
- Agent Skills : https://agentskills.io
- MCP, AGENTS.md et goose sont gérés depuis décembre 2025 par l'Agentic AI Foundation (Linux Foundation).

Source : Gloaguen et al. (2026), évaluation sur 138 dépôts réels ; Linux Foundation, 9 décembre 2025.

MCP : brancher des outils quand c'est utile



Ce que c'est

standard ouvert d'Anthropic (confié à l'Agentic AI Foundation en décembre 2025) pour relier un assistant IA à des outils, données et workflows via une architecture client-serveur ; idée clé : "brancher une fois, utiliser partout".



Dans ces options

Claude Code, Codex, OpenCode, Cline, Antigravity CLI : tous compatibles MCP (serveurs locaux ou distants).



Quand l'utiliser

- donner accès à une doc interne sans copier-coller,
- déclencher un script de build/QA,
- interroger une petite API ;

Non obligatoire pour le TP. 🙂

Pour aller plus loin :

- Documentation MCP officielle : https://modelcontextprotocol.io/
- FastMCP (pour faire ses propres MCP) : https://github.com/jlowin/fastmcp
- Exemples de serveurs MCP : https://github.com/modelcontextprotocol/servers
- L’incontournable Smithery : https://smithery.ai/

Best Practices : planification et process

01

Planification initiale

Définissez clairement vos objectifs, contraintes techniques et critères de succès avant de solliciter l'IA. Un cahier des charges précis = résultats de qualité.

Et maintenez le CDC et les docs techniques à jour en permanence (peut-être fait de manière automatique sur Claude Code grâce à CLAUDE.md, ou AGENTS.md pour les autres agents !)

02

Prompts réutilisables

Créez une bibliothèque de prompts testés pour les tâches récurrentes : génération de tests, refactoring, documentation, debugging. Gagnez du temps et de la cohérence. Et même mieux : **faites-en des skills !**

03

Gestion du contexte

Maintenez une "memory bank" avec les décisions architecturales, conventions de code et contraintes projet. Injectez ce contexte dans chaque session pour des réponses pertinentes. Et malgré la capacité d'auto-compression automatique du contexte de Claude Code, **favorisez les nouvelles sessions fraîches** quand une tâche s'avère trop lourde !

04

Documentation continue

Journalisez systématiquement : CDC initial, bugs complexes résolus, grosses features implémentées. Cette documentation devient votre référence et votre contexte de session en session.

Best Practices : sécurité et qualité



Backups multiples

Système à trois niveaux : local (copies régulières), Git (commits fréquents), Time Machine ou équivalent. Vérifiez régulièrement que vos backups fonctionnent. En auto mode, Git est obligatoire.



Anti-régression

Utilisez des checksums ou tests automatisés pour détecter les régressions introduites par l'IA. **Ne faites jamais confiance aveuglément aux modifications proposées !!!** Faites écrire les tests avant le code.



Sollicitations externes

Pour les décisions critiques, demandez une seconde opinion à une autre IA ou un sous-agent spécialisé. La validation croisée révèle souvent des angles morts.



💡 **Astuce pro :** Créez des sous-agents spécialisés (sécurité, performance, accessibilité) pour auditer le code généré par votre agent principal.

Pièges à éviter absolument

Le syndrome du "ça compile donc c'est bon"

L'IA génère du code syntaxiquement correct mais logiquement défaillant. Testez systématiquement, ne vous fiez pas aux apparences.

La perte de compréhension

Quand vous ne comprenez plus votre propre code, vous perdez le contrôle. Exigez des explications, documentez, simplifiez.

L'over-engineering assisté par IA

Les LLM adorent suggérer des solutions complexes. Gardez le principe KISS en tête : la simplicité est une vertu.

Négliger la sécurité

L'IA peut introduire des vulnérabilités subtiles (injection SQL, XSS, fuites de données). Auditez toujours le code sensible. Vérifiez aussi que chaque paquet suggéré existe vraiment (slopsquatting) et méfiez-vous des instructions cachées dans les fichiers lus par l'agent (injection de prompt).

Ignorer les licences

Le code suggéré peut être inspiré de sources protégées. Vérifiez les licences, surtout pour du code commercial.

Tests réguliers et bissect Git

Les "régressions surprises" sont légion en vibe-coding : l'IA peut introduire des bugs subtils dans des parties du code apparemment non liées. Testez régulièrement l'ensemble de votre solution, pas seulement la partie modifiée. Utilisez la méthode du bissect Git pour identifier rapidement le commit responsable d'une régression : c'est un gain de temps énorme quand tout semblait fonctionner parfaitement la veille !

Veille technologique et synthèse

Comment faire sa veille efficacement

- **Newsletters ciblées :** TLDR, Hacker News Digest, AI Weekly
- **Twitter/X :** suivez les créateurs d'outils IA et les early adopters
- **Discord/Slack/Whatsapp :** communautés actives (Claude, OpenAI, local dev)
- **GitHub Trending :** découvrez les nouveaux outils émergents
- **Expérimentez :** testez un nouvel outil chaque mois



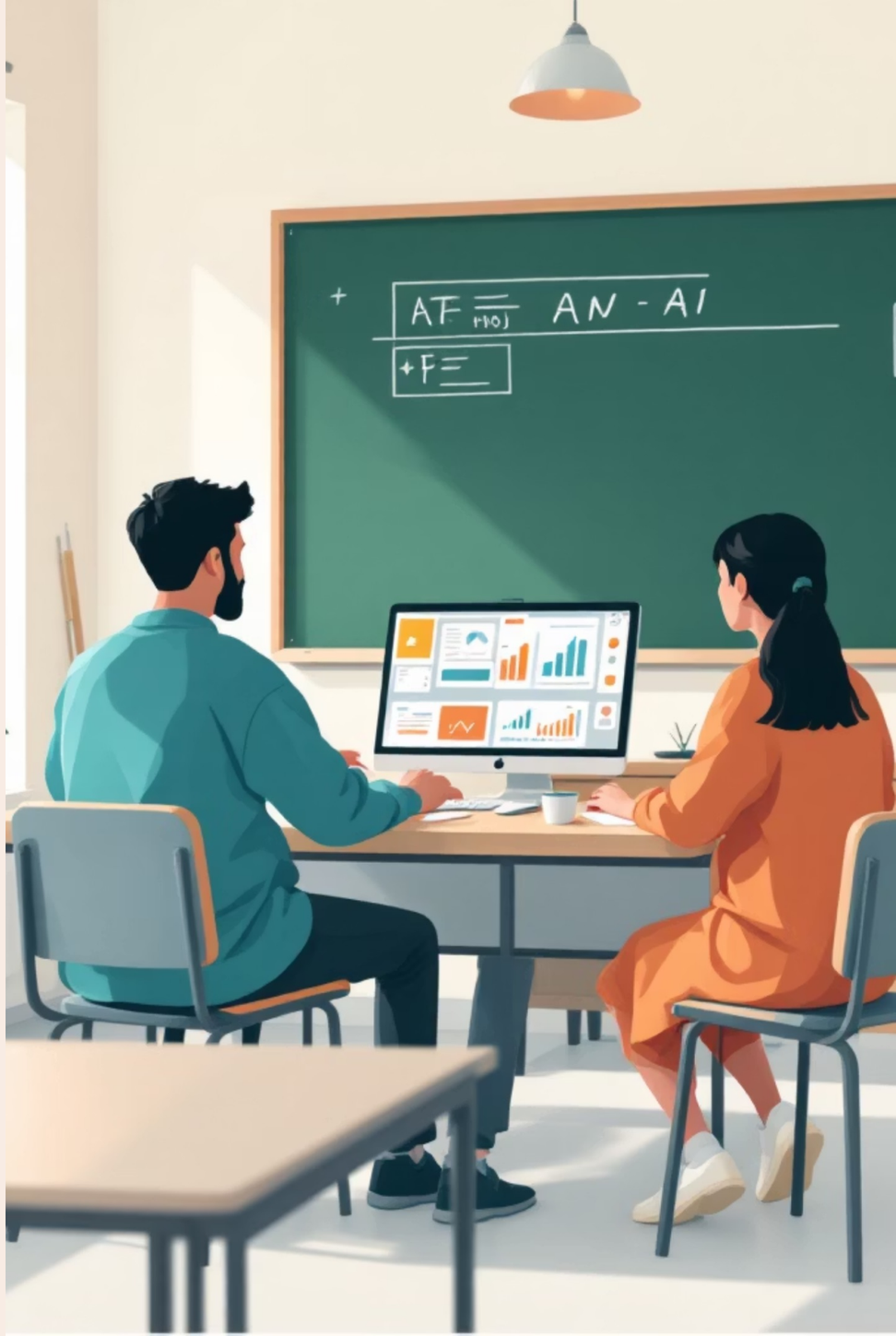
Points clés à retenir

Le vibe coding est un multiplicateur de productivité puissant qui nécessite discipline et méthode. Planifiez, documentez, sauvegardez. Protégez votre santé mentale en établissant des limites claires. Restez critique face aux suggestions de l'IA et maintenez votre compréhension du code. Le vibe coding sert à prototyper ; pour livrer, passez à l'agentic engineering.

À vous de jouer !

Après avoir exploré les stratégies et les bonnes pratiques, il est temps de **passer à l'action**. Mettez en application les concepts appris et **créez votre propre projet avec l'IA**.

C'est l'occasion d'expérimenter, de prototyper et de **transformer la théorie en résultats concrets !**



TP : créer votre application mobile avec l'IA

Objectif du projet :

Développer une application mobile innovante en utilisant les techniques de vibe coding apprises. Votre application doit exploiter au minimum 3-4 capteurs ou fonctionnalités natives de l'appareil mobile.

Capteurs et fonctionnalités à intégrer (choisir au moins 3-4) :

- **Caméra :** Capture photo/vidéo, reconnaissance d'objets, réalité augmentée
- **Microphone :** Enregistrement audio, reconnaissance vocale, analyse sonore
- **Accéléromètre :** Détection de mouvement, compteur de pas, jeux basés sur l'inclinaison
- **Gyroscope :** Orientation 3D, stabilisation, contrôles gestuels
- **GPS/Géolocalisation :** Cartographie, géofencing, tracking de parcours
- **Capteur de luminosité :** Adaptation automatique de l'interface
- **Capteur de proximité :** Interactions sans contact
- **Boussole :** Navigation, réalité augmentée géolocalisée

TP : Évaluation et Livrables

Critères d'évaluation

1	**Workflow personnel documenté (20%)** Votre méthodologie de vibe coding, outils utilisés, process de développement
2	**Innovation technique (25%)** Utilisation créative des capteurs, originalité de l'application
3	**Qualité du code (20%)** Architecture, lisibilité, bonnes pratiques
4	**Tests et déploiement (20%)** Tests sur device réel, compilation réussie, fonctionnalités opérationnelles
5	**Documentation (15%)** README, guide d'utilisation, retour d'expérience

Livrables attendus

- Code source complet avec historique Git
- Application compilée et testée sur mobile
- Documentation de votre workflow vibe coding + exemples de prompts clés + un AGENTS.md
- Démonstration fonctionnelle (5-10 minutes)

Conseils :

Commencez par définir votre concept, puis établissez votre workflow de développement avec l'IA. Testez régulièrement sur device réel. N'hésitez pas à itérer et à documenter vos découvertes !

Stack libre : idéalement une techno que vous maîtrisez déjà (typiquement Java/Kotlin), pour bien relire le code de l'agent. Ce n'est pas une obligation.